SIMATS ENGINEERING
ASSESSMENT TOOL 2: Scenario-Based

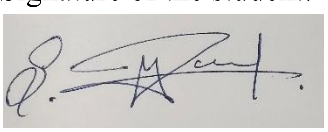
COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Name	MAGESH.S
Register Number	192525002

COURSE OUTCOME COVERED
CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 25**
Question Count: 5

Code of Conduct
I **MAGESH. S (192525002)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student:



Assessment Tasks Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering.
   Design a cloud architecture using scalability and caching techniques.
   Explain how load balancing and auto-scaling can ensure smooth streaming.
   Discuss cost optimization strategies for such high-demand workloads.

2. A fintech startup is deploying applications using containers across multiple cloud providers.
   They face challenges in maintaining consistency and managing deployments.
   Propose a solution using container orchestration and multi-cloud strategies.
   Explain how service discovery and scaling can be handled efficiently.
   Highlight the benefits and risks of a multi-cloud deployment model.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud.
The organization needs a robust backup and disaster recovery strategy.
Design a solution using cloud-native backup, replication, and versioning.
Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved.
Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data. Some applications must remain on-premises due to compliance requirements.
Design a hybrid architecture that ensures secure communication between environments.
Explain how identity management and encryption should be implemented.
Discuss challenges in managing hybrid cloud environments.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application.
The system must continue operating even if one region goes down.
Propose a multi-region deployment strategy in the cloud.
Explain how data replication and failover mechanisms should be configured.
Discuss monitoring and alerting strategies for proactive issue detection.

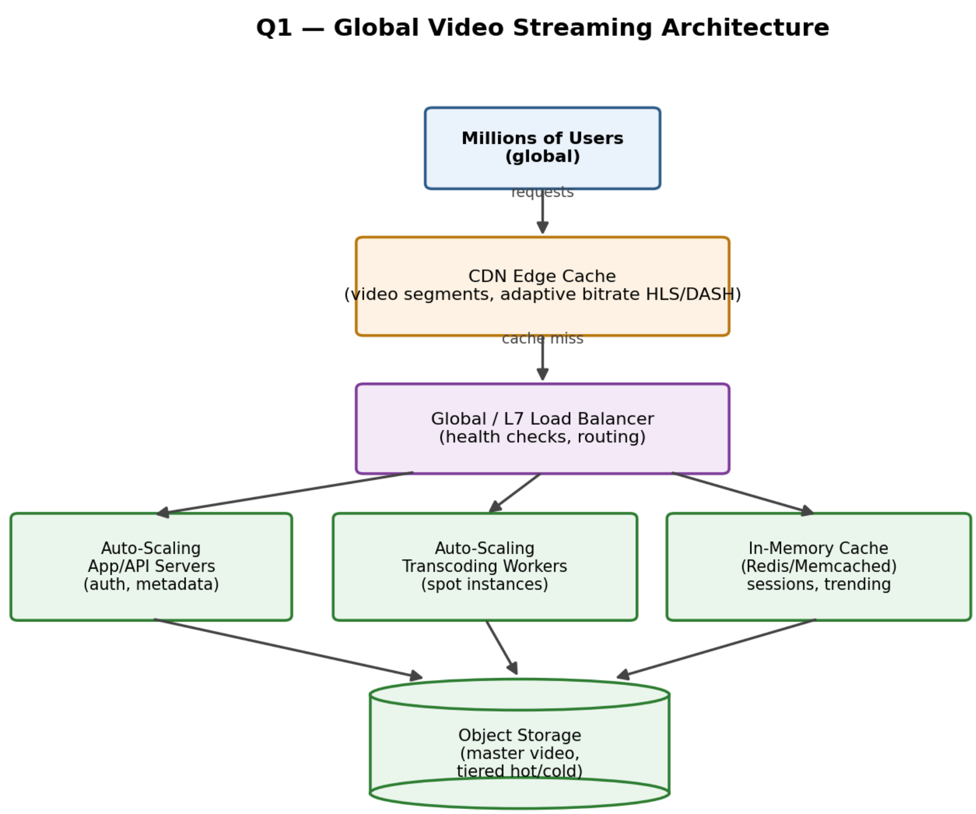
Scenario-Based Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and wellorganized response	Generally clear	Somewhat unclear	Poorly presented

1. **A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering. Design a cloud architecture using scalability and caching techniques. Explain how load balancing and auto-scaling can ensure smooth streaming. Discuss cost optimization strategies for such high-demand workloads.**

Cloud Architecture Design

- Content delivery: Utilize a global CDN (CloudFront, Akamai, Cloudflare) to deliver video segments to user endpoints via edge locations. This will reduce latency significantly and eliminate origin load.
- Adaptive streaming: Encode video into multiple streams of different bit rates (HLS/DASH) so that the player stream adapts to the user's available bandwidth.
- Compute tier: Stateless microservices for video transcoding, recommendations, and authentication are placed into auto-scale groups of virtual machines/containers managed by a load balancer. This prevents any single server from becoming a bottleneck.
- Caching tiers: Multilayered caching where a CDN edge layer caches video segments; an in-memory cache (Redis/Memcached) stores metadata, session tokens, and trending content; and origin-level cache protects backend storage from read loads.
- Storage: Object storage (S3 class) for master videos with lifecycle tiering (hot for recent release, cold or archival for past catalog).



Load balancing and auto-scaling for seamless streaming

Layer 7 load balancers distribute incoming traffic to healthy backend instances; they do health checks and bypass unhealthy backend instances. Auto-scaling based on metrics (CPU utilization, number of requests, number of concurrent connections) adds compute instances during peak hours (when, for example, new episodes drop or live streams occur) and scales them back later, keeping the performance elastic. With edge caching through CDN, most of the video data does not hit the origin, and so the load balancer and auto-scaler only deal with API/auth/metadata traffic and cache misses.
Strategies for cost optimization

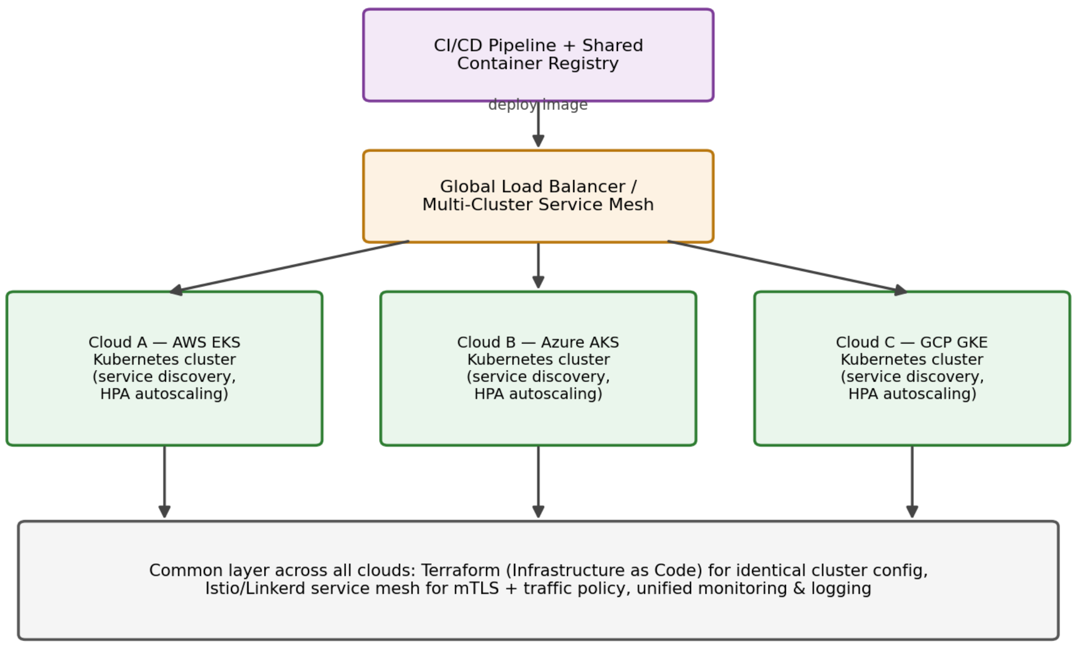
- Spot or preemptible VMs for batch processing tasks such as transcoding, reserved/committed instances for the base load.
- Storage tiering: move rarely accessed video content into cheaper cold storage automatically.
- Cache the video content at the CDN edge layer to reduce egress and compute costs.
- Set appropriate auto-scaling limits to not over-provision during moderate traffic periods; use serverless functions for rare bursty tasks (such as thumbnail generation).

2. **A fintech startup is deploying applications using containers across multiple cloud providers. They face challenges in maintaining consistency and managing deployments. Propose a solution using container orchestration and multi-cloud strategies. Explain how service discovery and scaling can be handled efficiently. Highlight the benefits and risks of a multi-cloud deployment model.**

Proposed solution: Container orchestration & multi-cloud approach

- Use Kubernetes for container orchestration because it is consistent in AWS EKS, Azure AKS, Google Cloud GKE, as well as on-premise clusters, providing the same deployment strategy, no matter what the underlying cloud.
- Packaging all services into containers with one CI/CD pipeline (one build, any deployment), using a single container repository that is replicated in all clouds.
- Infrastructure as code (Terraform) for defining clusters and network settings consistently in all clouds, avoiding any deviations in manual configuration.
- A service mesh (Istio/Linkerd) for traffic management, mutual TLS and observability consistently in all clusters.

Q2 — Multi-Cloud Container Orchestration (Fintech)



Service discovery and scaling

The internal service discovery (using DNS-based service names) of Kubernetes helps microservices discover each other consistently regardless of whether the microservices are deployed in Cloud A or Cloud B, without getting entangled with the specifics of each provider's networking. In addition, HPA will scale up or scale down replicas of containers using CPU or memory or custom metrics within each cluster, while the cluster autoscaler will scale up or scale down nodes depending on the needs of the cluster.

Benefits and Risks of Multi-Cloud Deployment

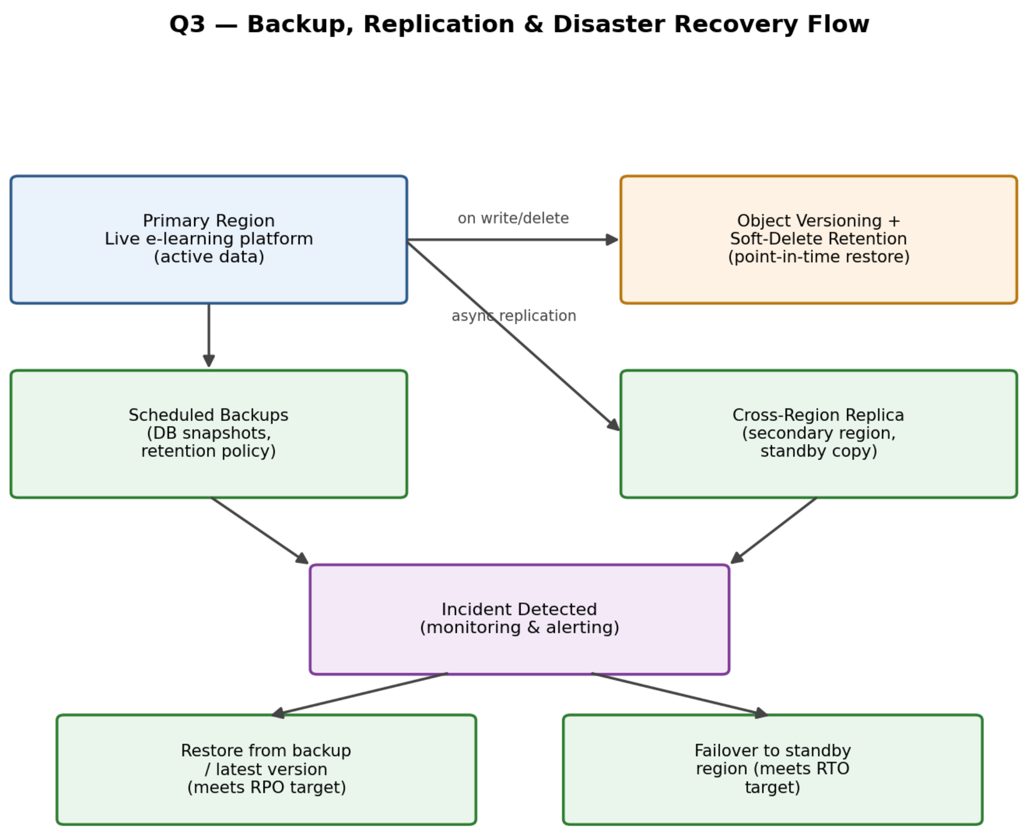
Benefits: no vendor lock-in, better resiliency (outage of one vendor won't affect the whole platform), ability to use the best or the cheapest service from a vendor per workload and, finally, ability to fulfill the regulatory data residency requirements.

Risks: operational complexity (several control planes, billing systems, security policies to coordinate), increased costs and latencies for inter-cloud networking, difficulties maintaining consistent configurations and compliance, and increased attack surface due to the reconciliation of identity and security policies between vendors, which is vital for a fintech company with sensitive financial data.

3. **An e-learning platform experiences data loss due to accidental deletion in the cloud. The organization needs a robust backup and disaster recovery strategy. Design a solution using cloud-native backup, replication, and versioning. Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved. Provide steps to ensure business continuity during failures.**

Backup, replication, and versioning design

- Enable object versioning on cloud storage (e.g., S3 versioning) so every file overwrite or deletion preserves prior versions, allowing point-in-time restore of accidentally deleted content.
- Configure automated, scheduled backups (database snapshots, file-system backups) with a defined retention policy (e.g., daily for 30 days, weekly for 6 months).
- Use cross-region replication so a copy of all critical data exists in a geographically separate region, protecting against regional failures in addition to accidental deletion.
- Apply the 3-2-1 backup rule: 3 copies of data, on 2 different storage media/types, with 1 copy off-site/off-platform.
- Enable soft-delete / retention locks on storage buckets so deletions are reversible within a grace window before being permanently purged.



Achieving RTO and RPO

Recovery Point Objective (RPO) — the maximum acceptable data loss — is achieved by setting backup/replication frequency tighter than the RPO target (e.g., continuous replication or 15-minute snapshot intervals for an RPO of 15 minutes). Recovery Time Objective (RTO) — the maximum acceptable downtime — is achieved by pre-provisioning a warm/hot standby environment (rather than restoring from cold backup only), automating the failover process, and regularly testing restore procedures so the actual recovery time is known and rehearsed, not theoretical.

Steps to ensure business continuity

1. Detect the incident (monitoring/alerting flags the deletion or outage).
2. Isolate the affected system to prevent further data loss or propagation.
3. Restore from the most recent valid backup/version, prioritizing the RPO target.
4. Fail over to the standby/replica environment if the primary is unavailable, meeting the RTO target.
5. Validate data integrity and application functionality before resuming full service.

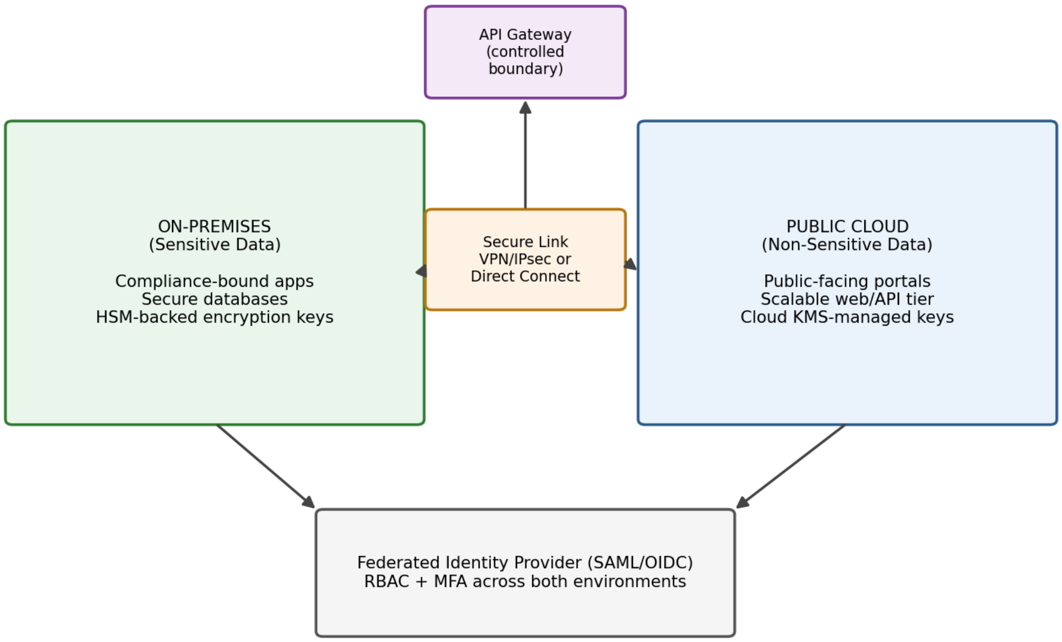
Conduct a post-incident review and update backup/DR runbooks accordingly

4. **A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data. Some applications must remain on-premises due to compliance requirements. Design a hybrid architecture that ensures secure communication between environments. Explain how identity management and encryption should be implemented. Discuss challenges in managing hybrid cloud environments.**

Hybrid architecture design

- Keep sensitive/regulated workloads on-premises (or in a private cloud) where compliance mandates data residency and direct control, while non-sensitive workloads (public-facing portals, non-confidential processing) run in the public cloud for scalability and cost efficiency.
- Connect the two environments via a secure, dedicated link — a VPN with IPsec encryption or a dedicated private connection (e.g., AWS Direct Connect / Azure ExpressRoute) — rather than routing sensitive traffic over the public internet.
- Use an API gateway as the controlled boundary between environments, so cloud-hosted applications only interact with on-prem systems through authenticated, well-defined APIs rather than direct database access.
- Apply network segmentation (separate VLANs/subnets, firewalls, and DMZs) so a breach in the public-cloud segment cannot directly reach on-prem sensitive systems.

Q4 — Hybrid Cloud Architecture (Government Agency)



Identity management and encryption

Deploy a federated identity system (SAML/OIDC) with a single on-prem or cloud-based IdP (e.g., Active Directory Federation Services) so users authenticate once and access both environments under a consistent identity and access policy. Enforce role-based access control (RBAC) and multi-factor authentication for any access crossing the on-prem/cloud boundary. Encrypt data in transit (TLS/IPsec across the hybrid link) and at rest (on-prem HSM-backed keys for sensitive data; cloud KMS-managed keys for cloud-hosted data), with strict key-management separation between environments.

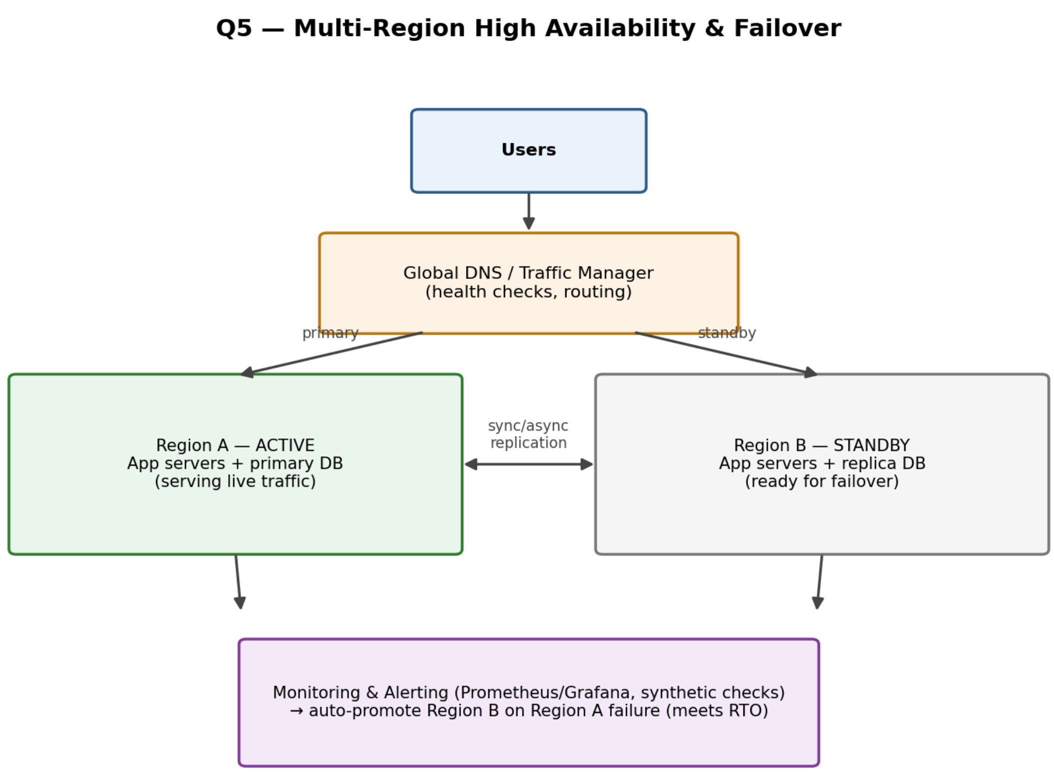
Challenges in managing hybrid cloud

- Maintaining consistent security policy and identity across two fundamentally different environments.
- Network latency and reliability of the link between on-prem and cloud, which can affect application performance.
- Increased operational complexity — two sets of tools, monitoring systems, and skill sets to manage.
- Compliance auditing across a split environment is harder, since evidence and logs must be reconciled from both sides.
- Data classification governance — consistently and correctly determining what data is allowed to move to the cloud versus what must stay on-prem.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application. The system must continue operating even if one region goes down. Propose a multi-region deployment strategy in the cloud. Explain how data replication and failover mechanisms should be configured. Discuss monitoring and alerting strategies for proactive issue detection.

Multi-region deployment strategy

- Deploy the application actively in at least two (ideally three) geographic regions, either in an active-active configuration (all regions serve live traffic) or active-passive (one primary, one hot standby ready to take over).
- Use a global DNS/traffic manager (e.g., Route 53 with health checks, Azure Traffic Manager) to route users to the nearest healthy region and automatically redirect traffic away from a failed region.
- Containerize/orchestrate the application (Kubernetes across regions) so the same deployment artifact runs identically in every region, avoiding configuration drift.



Data replication and failover mechanisms

Use a multi-region database with synchronous replication within a region (for consistency) and asynchronous cross-region replication (to avoid latency penalties on writes), accepting a small, defined

RPO for the cross-region copy. Configure automated failover: health checks continuously probe each region; if the primary region fails, DNS/traffic manager and database failover promote the standby region to primary within a defined RTO, with minimal manual intervention. Regularly run failover drills (chaos engineering / game days) to verify the failover process actually works under real conditions, not just on paper.

Monitoring and alerting for proactive detection

- Implement distributed monitoring (Prometheus/Grafana, CloudWatch, Datadog) collecting metrics from every region — latency, error rates, resource utilization, replication lag.
- Set threshold-based and anomaly-based alerts (e.g., error rate spike, replication lag exceeding RPO target, health-check failures) that page the on-call team before users notice degradation.
- Use synthetic monitoring (simulated user transactions from multiple locations) to detect regional issues proactively, and maintain a status dashboard so both engineering and stakeholders have real-time visibility into system health across all regions.